Ingeniería del Software II -Tema 2.

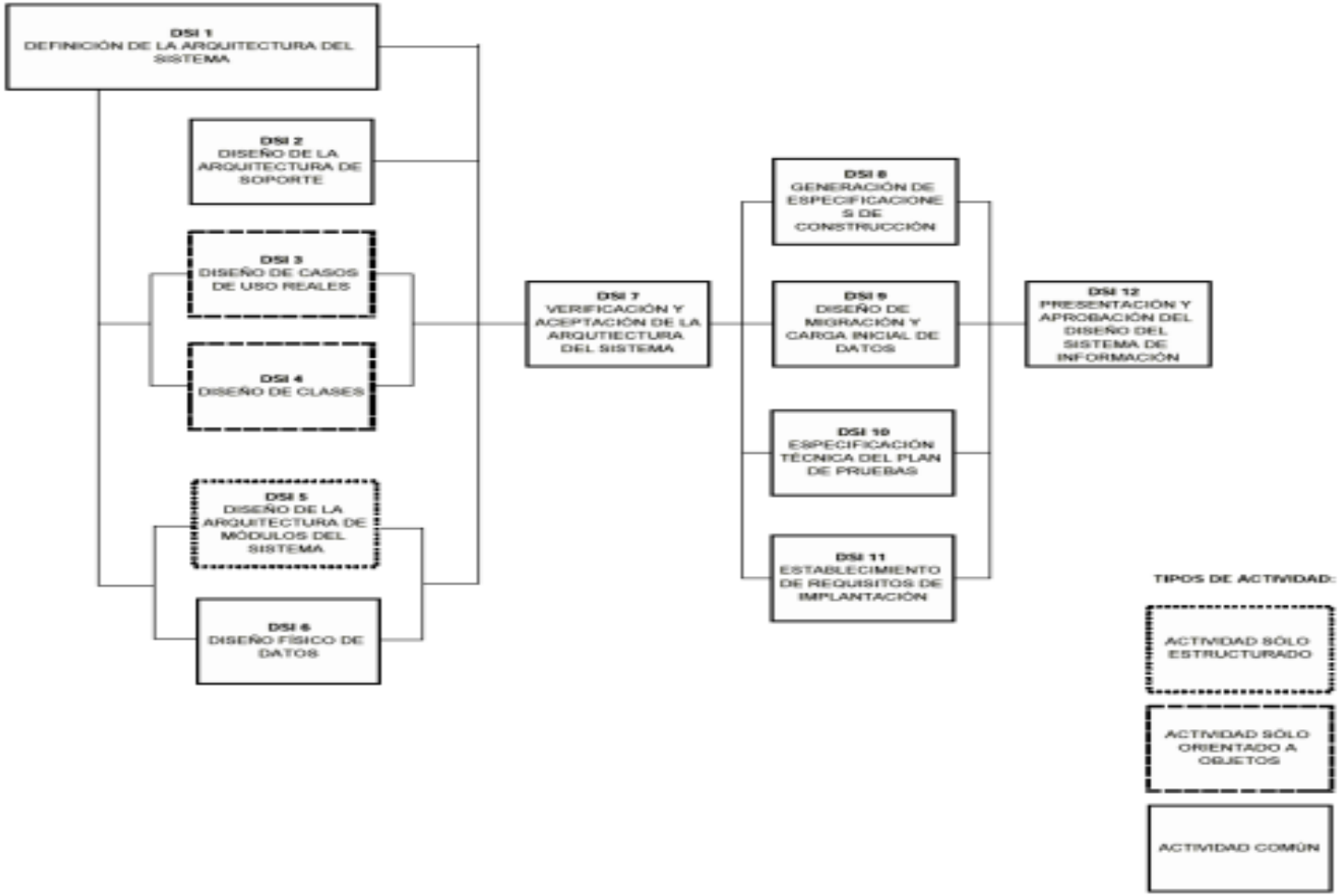
Diseño de Software. Diseño de Interfaz de Usuario.

- Del Análisis al Diseño. Conceptos Principales del Diseño. Diseño de Interfaz de Usuario. Diseño de Componentes. Independencia Funcional. El procedimiento de derivación. Análisis de transacciones y transformación. Evaluación y refinamiento: criterios, acoplamiento y cohesión. Documentación del Diseño. Diseño Ux/ Ui. Técnicas y consideraciones

Introducción

- Esquema general de los procesos que intervienen en el diseño de un sistema.

Fiura: Diseño del Sistema de Información (Proceso DSI)



Diseño de la Arquitectura de módulos del Sistema

Objetivo: para cada uno de los subsistemas se diseña la estructura modular de los procesos que lo integran.

Punto de partida: modelo de procesos obtenido en ASI y catálogo de requisitos.

Se realiza el diseño detallado de la interfaz de usuario, de pantalla e impresa.

La interfaz de usuario debe corresponderse con la estructura modular.

El Proceso de Diseño.

- “El proceso de aplicar distintas técnicas y principios con el propósito de definir un dispositivo, un proceso o un sistema con suficiente detalle como para permitir su realización física”.
- Proceso iterativo a través del cual se traducen los requisitos en una representación del software.
- **Diseño de datos**. Transforma el modelo del dominio de la información del análisis en las estructuras de datos necesarias para la implementación. ***Esquema Lógico de Datos “Modelo Relacional”***.
- **Diseño arquitectónico**. Estructura modular del programa/aplicación.
- **Diseño de interfaz**. Interfaces del sw. con otros sistemas y con los usuarios.
- **Diseño procedimental**. Descripción procedimental de los componentes del sw.

El Proceso de Diseño.

Definición de la arquitectura del sistema.

- Se define la arquitectura general del SI:
  - Particiones físicas
  - Descomposición lógica en subsistemas de diseño
  - Ubicación de cada subsistema en cada partición
  - Especificación detallada de la infraestructura tecnológica

Diseño de la arquitectura de módulos del sistema.

- Objetivo: para cada uno de los subsistemas se diseña la estructura modular de los procesos que lo integran.

- Pto. de partida: modelo de procesos obtenido en ASI y catálogo de requisitos.
- Técnica: Diagrama de Clases de Diseño
- Se realiza el diseño detallado de la interfaz de usuario, de pantalla e impresiones.
- El interfaz de usuario debe corresponderse con la estructura modular.

Diseño físico de datos.

- Objetivo: definir la estructura física de datos que utilizará el sistema, a partir del modelo lógico de datos normalizado o del modelo de clases
- “Paso a tablas”, si se usa un SGBDR.
- Se analizan los caminos de acceso a los datos persistentes, por parte de cada módulo, con el fin de mejorar los tiempos de respuesta y optimizar recursos.

Verificación y aceptación de la arquitectura del sistema.

- Objetivos: garantizar la calidad de las especificaciones del diseño y su viabilidad:
  - Verificación de la calidad técnica de cada modelo
  - Aseguramiento de la coherencia entre varios modelos
  - Aceptación del diseño por parte de Explotación y Sistemas

Generación de especificaciones de construcción.

- A partir del diseño anterior, se generan las especificaciones para la construcción del SI, incluyendo:
  - Especificación del entorno de construcción:
    - herramientas, compiladores, generadores de código, etc.
  - Descripción de componentes
  - Especificación detallada de componentes
    - normalmente en pseudocódigo
  - Especificación de la estructura física de datos
    - definición y creación de los elementos del modelo físico de datos con el DDL del SGBD escogido

Especificación técnica del plan de pruebas.

- Se especifica en detalle el plan de pruebas del SI, para los niveles de prueba:
  - Pruebas unitarias
  - Pruebas de integración
  - Pruebas de implantación
  - Pruebas de aceptación
- Se especifica el entorno de las pruebas
- Se definen los casos de prueba

El Proceso de Diseño.

- Estrategias de diseño - *Tipos de Esquemas*:
  - Análisis de transformaciones
  - Análisis de transacciones

- Se dispone de:
  - ***Las entradas*** que suministran al sistema las ***entidades externas***.
  - *Las salidas* aportadas por el sistema a dichas ***entidades externas***.
  - Las funciones descompuestas que se han de realizar en ese sistema.
  - El ***esquema lógico*** de datos del sistema.

El Proceso de Diseño.

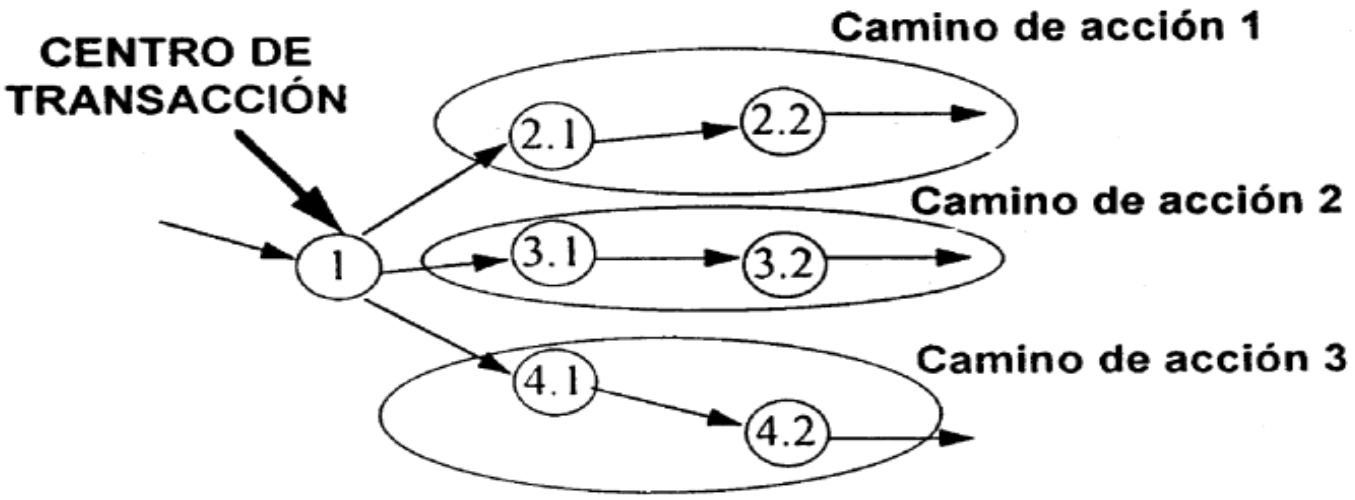
- Se basa en los siguiente *Principios*:
  - Abstracción
  - Modularidad
  - Encapsulamiento y Ocultamiento de información

Puede ser necesario refinar el diseño de funcionalidades.

- Dos estrategias:
- Análisis de transformaciones.
- Análisis de transacciones.

- Importante: diseñar de forma que:
  - Los módulos de nivel superior toman las decisiones de ejecución (coordinan).
  - Los de nivel inferior realizan la mayor parte del trabajo de entrada, de cálculo y de salida.

Análisis de Transacción. 1º Nivel de Factorización.



Criterios de Validación de Calidad

Es vital que esa partición sea hecha de tal manera que los módulos sean tan independientes como sea posible y que cada módulo ejecute una única función. Para que los diseños tengan esas cualidades, son necesarios algunos criterios de medición que permitan clasificar y mejorar los diagramas de estructura. A continuación se describen los criterios utilizados para mejorar un diseño.

Acoplamiento.

El acoplamiento entre módulos clasifica el grado de independencia entre pares de módulos de un DE. El objetivo es minimizar el acoplamiento, es decir, maximizar la independencia entre módulos. A pesar de que el acoplamiento, es un criterio que clasifica características de *una invocación (una relación existente entre dos módulos), será usado* para clasificar.

Un bajo acoplamiento indica un sistema bien particionado y puede obtenerse de tres maneras:

• *Eliminando relaciones innecesarias: Por ejemplo, un módulo puede recibir algunos* datos, innecesarios para él, porque debe enviarlos para un módulo subordinado.

• *Reduciendo el número de relaciones necesarias: Cuanto menos conexiones existan* entre módulos, menor será la posibilidad del efecto en cadena (un error en un módulo aparece como síntoma en otro).

• *Debilitando la dependencia de las relaciones necesarias: Ningún módulo se tiene que* preocupar por los detalles internos de implementación de cualquier otro. Lo único que tiene que conocer un módulo debe ser su función y las cuplas de entrada y salida (cajas negras).

Cohesión

Otro medio para evaluar la partición en módulos (además del acoplamiento) es observar cómo las actividades de un módulo están relacionadas unas con otras; este es el criterio de cohesión. Generalmente el tipo de cohesión de un módulo determina el nivel de acoplamiento que tendrá con otros módulos del sistema.

Cohesión es la medida de intensidad de asociación funcional de los elementos de un módulo. Por elemento debemos entender una instrucción, o un grupo de instrucciones o una llamada a otro módulo, o un conjunto de procedimientos o funciones empaquetados en el mismo módulo.

El objetivo del diseño estructurado es obtener módulos altamente cohesivos, cuyos elementos estén fuerte y genuinamente relacionados unos con otros.

Por otro lado, los elementos de un módulo no deberían estar fuertemente relacionados con elementos de otros módulos, porque eso llevaría a un fuerte acoplamiento entre ellos.

Descomposición (Factoring)

La descomposición es la separación de una función contenida en un módulo, para un nuevo módulo. Puede ser hecha por cualquiera de las siguientes razones.

Reducir el tamaño del módulo

La descomposición es una manera eficiente de trabajar con módulos grandes. Un buen tamaño para un módulo es alrededor de media página (30 líneas). Ciertamente, toda codificación de un módulo debería ser visible en una página (60 líneas).

La cantidad de líneas no es un patrón rígido, otros criterios para determinar cuándo es conveniente terminar de realizar la descomposición, son los siguientes:

• *Funcionalidad: Terminar de realizar la descomposición cuando no se pueda encontrar* una función bien definida. No empaquetar líneas de código dispersas, de otros módulos, porque probablemente juntas podrán formar módulos con mala cohesión.

- *Complejidad de Interfaz: Terminar de realizar la descomposición cuando la interfaz* de un módulo *es tan compleja como el propio módulo. Un módulo de mil líneas es* muy confuso, más mil módulos de una línea son aún más confusos.

Descomposición (Factoring)

Hacer el sistema más claro

La descomposición no debería ser hecha de una manera arbitraria, los módulos resultantes de la descomposición de un módulo deben representar sub-funciones del módulo de más alto nivel en el DE.

En una descomposición no se debe preocupar por conceptos de programación. Si una sub-función, presentada como un módulo separado permite una mejor comprensión del diseño, puede ser subdividida, aún cuando, en una implementación, el código del módulo sea programado dentro del módulo jefe.

Minimizar la duplicación de código

Cuando se reconoce una función que puede ser reutilizada en otras partes del DE, lo más conveniente es convertirla en un módulo separado. Así, se puede localizar más fácilmente las funciones ya identificadas y evitar la duplicación del mismo código en el interior de otro módulo. De esta manera, los problemas de inconsistencia en el mantenimiento (si esa función debe ser modificada) pueden ser evitados, y se reduce el costo de implementación.

Separar el trabajo de la ‘administración’

Un administrador o gerente de una compañía bien organizada debería coordinar el trabajo de los subordinados en lugar de *hacer el trabajo. Si un gerente hace el trabajo de los* subordinados no tendrá tiempo suficiente para coordinar y organizar el trabajo de los subordinados y, por otro lado, si hace el trabajo los subordinados no serían necesarios. Lo mismo se puede aplicar al diseño del DE, relacionado a los módulos de *Trabajo (edición,* cálculo, etc.) y a los módulos de *Gerencia (decisiones y llamadas para otros módulos).*

El resultado de este tipo de organización es un sistema en el cual los módulos en los niveles medio y alto de un DE son fáciles de implementar, porque ellos obtienen el trabajo hecho por la manipulación de los módulos de los niveles inferiores. La separación del trabajo de la *administración mejora la mantenibilidad del diseño.* Una alteración en un sistema es: un cambio de control o un cambio de trabajo, pero raramente ambos.

Crear módulos más generales

Otra ventaja de la descomposición es que, frecuentemente, se pueden reconocer módulos más generales y, así, más útiles y reutilizables en el mismo sistema y, además, pueden ser generadas bibliotecas de módulos reutilizables en otros sistemas.

Documentación.

La documentación del diseño es vital para garantizar su legibilidad y correcto entendimiento en la etapa de construcción o codificación, además de permitir detectar errores de consistencia.

Debe entenderse que el diseño debe ser perfectible por personas ajenas a quienes lo construyeron, por lo que la documentación no puede faltar.

Por otro lado, no hay que olvidar que el diseño constituye la especificación de la etapa subsiguiente.

Para documentar el diseño se tienen que documentar todos los elementos que aparecen en los diagramas de flujos de datos y diagramas de estructuras, esto es: Terminadores, almacenes de datos, flujos de datos y procesos.

Bibliografía

 Pressman, Roger S. Ingeniería Software Connect. Edición 2021. Mc Graw Hill Interamericana. ISBN: 9781456287726.

Ingeniería del Software, Sommerville, Ian · 9na ed. Pearson. 2011